

SYSTEM PROGRAMMING

Topic 1: Process Management

1. Write a program to create a child process using `fork()`. The parent process should display even numbers from 1 to 100, while the child process displays odd numbers from 1 to 100.
2. Write a program where a parent process creates two child processes. The first child computes the factorial of a given number and the second child generates the Fibonacci series up to N terms.
3. Write a program where the parent process creates three child processes to calculate the sum, average, and maximum of an integer array. The parent should wait for all child processes and display their exit status.
4. Write a program to create a process tree consisting of one parent, two children, and four grandchildren. Display the PID and PPID of every process.
5. Write a program where a child process executes the `ls -l` command using the `exec()` family of system calls while the parent waits for its completion.
6. Write a program where the parent process executes the `date` command and the child process executes the `cal` command using `exec()`.
7. Write a program to demonstrate the creation of a zombie process and display the process table to verify its existence.
8. Write a program to demonstrate the creation of an orphan process and display the parent process ID before and after orphan creation.
9. Write a program where the parent creates four child processes, each calculating the square of a different number. The parent collects and displays the completion status of each child.
10. Write a program in which a parent process creates a child process. The child repeatedly prints its PID until the parent terminates it using a signal.
11. Write a program to demonstrate the use of `wait()` and `waitpid()` by creating multiple child processes with different execution times.
12. Write a program where the parent process creates a child that executes the `pwd` command and another child that executes the `whoami` command simultaneously.
13. Write a program where five child processes are created. Each child should display its PID, PPID, and process creation order.
14. Write a program to demonstrate process replacement using `execl()`, `execv()`, and `execvp()` by executing three different Linux commands.

15. Write a program where a parent process creates two child processes. One child counts vowels in a string while the other counts consonants.
16. Write a program to demonstrate process synchronization by ensuring that three child processes execute sequentially in the order Child1 → Child2 → Child3.
17. Write a program where a child process calculates the multiplication table of a given number while the parent simultaneously calculates the square table of another number.
18. Write a program where the parent creates two child processes. The first child sorts an array in ascending order, while the second child sorts another array in descending order.
19. Write a program to create a child process that executes another C program using `exec()`, while the parent process measures the execution time.
20. Write a program that demonstrates process creation using `fork()` recursively to generate a binary process tree of three levels.

Topic 2: Linux File Management

21. Write a program to create a file named `student.txt`, write five student records into it, and display the contents.
22. Write a program to copy the contents of one file into another using only Linux system calls (`open()`, `read()`, `write()`, and `close()`).
23. Write a program to append employee details entered by the user into an existing file without deleting previous records.
24. Write a program to count the total number of characters, words, and lines present in a text file.
25. Write a program to display the contents of a file in reverse order using `lseek()`.
26. Write a program to compare two text files and display whether they are identical or different.
27. Write a program to merge the contents of two files into a third file using Linux system calls.
28. Write a program to split a large text file into two equal files.
29. Write a program to create a file and display its inode number, permissions, and size using Linux system calls.
30. Write a program to rename an existing file and verify the operation.
31. Write a program to implement file locking where one process writes continuously into a file while another process waits until the lock is released.

32. Write a program where two processes simultaneously write different data into the same file using advisory file locking.
33. Write a program to replace every occurrence of a given word in a text file with another word.
34. Write a program to display the last 10 lines of a file without using Linux shell commands.
35. Write a program to search for a specific string in a file and display the line numbers where it occurs.
36. Write a program to encrypt the contents of a text file using a simple Caesar Cipher and store the encrypted text in another file.
37. Write a program to decrypt an encrypted text file created using Caesar Cipher.
38. Write a program to read student records from a file, sort them based on marks, and write the sorted records into another file.
39. Write a program where one process continuously appends data to a file while another process continuously reads the updated contents.
40. Write a program to create a sparse file using lseek() and verify its size and allocated disk blocks.

Topic 3 – Unnamed Pipes

41. Write a program where a parent process sends an integer to its child process through an unnamed pipe. The child should compute its factorial and send the result back to the parent using another unnamed pipe.
42. Write a program where the parent sends an array of integers through an unnamed pipe. The child process should sort the array in ascending order and return the sorted array.
43. Write a program to demonstrate two-way communication between parent and child processes using two unnamed pipes.
44. Write a program where the parent sends a string through a pipe. The child process should reverse the string and return it to the parent.
45. Write a program where the parent sends a file name through a pipe. The child process should count the number of lines in the file and send the result back.
46. Write a program where a child process sends the multiplication table of a given number to the parent through an unnamed pipe.
47. Write a program where the parent sends N integers to the child. The child should calculate the average and transmit the result back.

48. Write a program where the parent sends employee details through a pipe. The child should display only employees whose salary exceeds ₹50,000.
49. Write a program where the parent sends a matrix to the child process. The child should compute its transpose and return it.
50. Write a program where the child process receives a sentence through a pipe and counts the number of vowels and consonants.
51. Write a program where the parent sends five filenames through a pipe one after another. The child should display the size of each file.
52. Write a program where one child writes data into a pipe and another child reads the data through the same pipe.
53. Write a program to implement the Linux command `ls | sort` using unnamed pipes.
54. Write a program to implement the Linux command `cat file.txt | wc -w` using unnamed pipes.
55. Write a program where the parent sends a student's record to the child. The child should calculate the grade and return it.
56. Write a program where the parent sends a sentence to the child. The child should convert all lowercase letters to uppercase and return the modified sentence.
57. Write a program where the parent sends two matrices through an unnamed pipe. The child should perform matrix addition and send the result back.
58. Write a program where the parent sends ten integers through a pipe. The child should identify the prime numbers and return only the prime numbers.
59. Write a program where a parent process creates two child processes. The first child sends data through a pipe and the second child reads and processes the received data.
60. Write a program to implement a mini calculator where the parent sends two numbers and an operator through a pipe, and the child returns the computed result.

Topic 4 – Named Pipes (FIFO)

61. Write two separate programs to implement communication between a FIFO writer and FIFO reader using named pipes.
62. Write a program where process P1 sends a student's details to process P2 through a FIFO. Process P2 should display the received details.
63. Write a program where process P1 sends an integer array through a FIFO. Process P2 should sort the array and display the result.

64. Write a program where process P1 sends a file name through a FIFO. Process P2 should display the contents of the file.
65. Write a client-server application using FIFO where the client sends two integers and the server returns their sum.
66. Write a program where process P1 continuously sends temperature readings through a FIFO and process P2 displays the average temperature after every five readings.
67. Write a program where one process writes employee records into a FIFO and another process stores them in a file.
68. Write a program where process P1 sends a string through a FIFO. Process P2 should determine whether the string is a palindrome.
69. Write a program where two independent processes exchange messages using two named pipes.
70. Write a FIFO-based calculator supporting addition, subtraction, multiplication, and division.
71. Write a program where process P1 sends five filenames through a FIFO. Process P2 should display the size of each file.
72. Write a program where process P1 sends a matrix through a FIFO. Process P2 should calculate its determinant.
73. Write a program where process P1 sends an array of integers. Process P2 should identify the largest and smallest elements.
74. Write a program where process P1 sends a sentence. Process P2 should count the number of words and characters.
75. Write a program where process P1 sends a student's marks through a FIFO. Process P2 should calculate the total, average, and grade.
76. Write a program where one process continuously writes sensor values into a FIFO while another process continuously monitors and displays them.
77. Write a program where two users communicate through a FIFO-based text chat application.
78. Write a program where one process sends a filename to another process using a FIFO, and the receiver copies the file contents into a new file.
79. Write a program where multiple writer processes send messages to a single FIFO reader process. The reader should display all received messages.
80. Write a program where process P1 sends ten integers through a FIFO. Process P2 should separate the even and odd numbers and display them independently.

Topic 5 – Message Queues

- 81.** Write two programs where Process P1 sends a text message to Process P2 using a System V Message Queue.
- 82.** Write a client-server application using Message Queues where the client sends two integers and an arithmetic operator, and the server returns the result.
- 83.** Write a program where three sender processes send student details to a single receiver process through a Message Queue.
- 84.** Write a program where Process P1 sends an integer array through a Message Queue. Process P2 should sort the array in ascending order and display the result.
- 85.** Write a program where Process P1 sends a string to Process P2 through a Message Queue. Process P2 should determine whether the string is a palindrome.
- 86.** Write a program where Process P1 sends a filename through a Message Queue. Process P2 should count the number of words in the file and send the count back.
- 87.** Write a program where multiple clients send requests to a server through a Message Queue. The server should display all received requests in the order of arrival.
- 88.** Write a program where Process P1 sends employee records through a Message Queue. Process P2 should display only employees whose salary exceeds ₹60,000.
- 89.** Write a program where two Message Queues are used for bidirectional communication between two processes.
- 90.** Write a program where Process P1 sends five integers through a Message Queue. Process P2 should calculate the average and send the result back.
- 91.** Write a program where Process P1 sends a matrix to Process P2. Process P2 should compute the transpose and return it through another Message Queue.
- 92.** Write a program where Process P1 sends student marks through a Message Queue. Process P2 should compute grades for all students.
- 93.** Write a program where Process P1 sends sensor readings through a Message Queue every second, while Process P2 continuously monitors and displays them.
- 94.** Write a program using message priorities (message types) to process urgent messages before normal messages.
- 95.** Write a program where Process P1 sends ten integers to Process P2. Process P2 should identify the prime numbers and return them.

96. Write a program where multiple producer processes send messages to a Message Queue and a single consumer process reads and displays them.
97. Write a program where Process P1 sends the contents of a text file through a Message Queue in fixed-size blocks. Process P2 should reconstruct the file.
98. Write a program to implement a Message Queue-based notification system where multiple processes send event messages to a monitoring process.
99. Write a program where Process P1 sends an encrypted message through a Message Queue. Process P2 should decrypt and display it.
100. Write a program to demonstrate the creation, communication, and deletion of a Message Queue after successful message transfer.

Topic 6 – Shared Memory

101. Write two programs where Process P1 writes a string into Shared Memory and Process P2 reads and displays the string.
102. Write a program where Process P1 writes an integer array into Shared Memory. Process P2 should calculate the sum and average.
103. Write a program where Process P1 writes a matrix into Shared Memory. Process P2 should compute its transpose.
104. Write a program where Process P1 writes student records into Shared Memory. Process P2 should display students scoring above 80%.
105. Write a program where Process P1 writes employee records into Shared Memory. Process P2 should calculate the average salary.
106. Write a program where two processes continuously update a common counter stored in Shared Memory.
107. Write a program where Process P1 writes a sentence into Shared Memory. Process P2 should count the number of vowels and consonants.
108. Write a program where Process P1 writes a filename into Shared Memory. Process P2 should display the file size.
109. Write a program where Process P1 writes ten integers into Shared Memory. Process P2 should separate even and odd numbers.
110. Write a program where Process P1 writes five floating-point numbers into Shared Memory. Process P2 should calculate the standard deviation.
111. Write a program where Process P1 writes a matrix into Shared Memory. Process P2 should perform matrix multiplication with another matrix.
112. Write a program where two processes exchange messages alternately using Shared Memory.
113. Write a program where Process P1 writes examination marks into Shared Memory. Process P2 should generate grades and class averages.

- 114. Write a program where one process continuously writes sensor readings into Shared Memory while another process continuously monitors them.
- 115. Write a program where Process P1 stores employee attendance records in Shared Memory. Process P2 should calculate total working hours.
- 116. Write a program where multiple processes update a common bank account balance stored in Shared Memory.
- 117. Write a program where Process P1 writes a list of filenames into Shared Memory. Process P2 should display the size of each file.
- 118. Write a program where Process P1 stores a paragraph in Shared Memory. Process P2 should count words, sentences, and characters.
- 119. Write a program to demonstrate a race condition using Shared Memory when two processes simultaneously update the same variable.
- 120. Write a program to exchange data between two processes using Shared Memory and ensure proper cleanup by removing the Shared Memory segment after execution.

Topic 7 – Semaphores

- 121. Write a program to synchronize two processes accessing a shared counter using a binary semaphore.
- 122. Write a program to solve the Producer–Consumer problem using Shared Memory and Semaphores.
- 123. Write a program where three processes access a common file. Ensure that only one process writes to the file at a time using semaphores.
- 124. Write a program to solve the Reader–Writer problem using semaphores.
- 125. Write a program where two processes print odd and even numbers alternately using semaphores.
- 126. Write a program where three processes increment a shared integer 1000 times each using semaphores to prevent race conditions.
- 127. Write a program where multiple processes access a printer resource. Use semaphores to ensure only one process prints at a time.
- 128. Write a program where Process P1 writes student records into Shared Memory and Process P2 reads them only after P1 completes the write operation using semaphores.
- 129. Write a program where four processes compete to update a shared bank account balance. Synchronize the updates using semaphores.
- 130. Write a program where two producer processes and one consumer process share a common buffer using semaphores.

131. Write a program to solve the Dining Philosophers problem using semaphores.
132. Write a program where three processes access a shared log file. Ensure that the log entries are written without overlap using semaphores.
133. Write a program where Process P1 generates random numbers and Process P2 processes them. Synchronize both processes using semaphores.
134. Write a program where two processes alternately write to a common file using semaphores.
135. Write a program where multiple processes share a common message buffer. Use semaphores to avoid simultaneous access.
136. Write a program to implement a bounded buffer of size five using counting semaphores.
137. Write a program where Process P1 writes examination marks into Shared Memory and Process P2 calculates grades only after receiving semaphore permission.
138. Write a program where five processes compete for two identical resources. Implement synchronization using counting semaphores.
139. Write a program where multiple processes update a common inventory database. Use semaphores to maintain data consistency.
140. Write a program to demonstrate the difference between binary and counting semaphores by protecting access to shared resources.

Topic 8 – Threads & Multithreading

141. Write a program to create three threads. Each thread should print its thread ID and execute a different task.
142. Write a multithreaded program where one thread calculates the sum of even numbers and another thread calculates the sum of odd numbers from 1 to N.
143. Write a program where one thread computes the factorial of a number while another thread computes the Fibonacci series.
144. Write a multithreaded program where two threads update/write to the file a.txt and one thread continuously reads the updated contents.
145. Write a program where two threads simultaneously update a shared counter. Protect the counter using a mutex.
146. Write a multithreaded program where one thread reads student records from a file while another thread sorts the records based on marks.
147. Write a program where three threads compute the minimum, maximum, and average values of an integer array.

148. Write a program where two threads perform matrix addition and matrix subtraction simultaneously.
149. Write a multithreaded program where one thread performs matrix multiplication while another thread computes the transpose of the same matrix.
150. Write a program where one thread continuously writes sensor values into a file and another thread continuously displays the latest value.
151. Write a program where four threads search different portions of an array for a given key. Display the position if the key is found.
152. Write a program where one thread encrypts a text file while another thread decrypts it into another file.
153. Write a program where two threads print odd and even numbers alternately using mutexes and condition variables.
154. Write a multithreaded program to implement the Producer–Consumer problem using POSIX threads and mutex locks.
155. Write a program where multiple threads update employee salary records stored in memory. Ensure synchronization using mutexes.
156. Write a program where one thread generates random numbers and another thread computes their average every five seconds.
157. Write a multithreaded program where three threads independently sort three arrays and the main thread merges the sorted arrays.
158. Write a program where multiple threads access a common file and record their thread IDs along with timestamps.
159. Write a multithreaded program where one thread monitors keyboard input while another thread continuously writes the received data into a log file.
160. Write a program where one thread reads a text file, another thread counts the words, and a third thread counts the characters. Display the results after all threads complete execution.

Topic 9 – TCP/IP Socket Programming

Basic Socket Communication

161. Write the **server-side** program in TCP/IP socket communication to receive a message from a client and display it.
162. Write the **client-side** program in TCP/IP socket communication to send a message to the server.
163. Write a TCP client-server program where the client sends two integers and the server returns their sum.
164. Write a TCP client-server application where the client sends a string and the server returns the reversed string.

165. Write a TCP client-server program where the client sends an integer array and the server returns the largest and smallest elements.

File Transfer

166. Write a TCP client-server application where the client sends the name of a text file and the server sends back its contents.

167. Write a TCP client-server application to transfer a text file from the client to the server.

168. Write a TCP client-server application where the client uploads an image file and the server stores it successfully.

String Processing

169. Write a TCP client-server application where the client sends a sentence and the server returns the number of words, vowels, and consonants.

170. Write a TCP client-server application where the client sends a paragraph and the server converts all lowercase letters to uppercase before returning it.

Multiple Clients

171. Write a multithreaded TCP server that handles multiple clients simultaneously.

172. Write a TCP chat application where one server communicates with two clients independently.

173. Write a TCP server that accepts messages from multiple clients and stores all received messages in a log file.

174. Write a TCP server where each connected client is assigned a unique client ID and receives a welcome message.

Applications

175. Write a TCP client-server calculator supporting addition, subtraction, multiplication, division, and modulus operations.

176. Write a TCP client-server application where the client sends student details and the server computes the grade.

177. Write a TCP client-server voting application where multiple clients cast votes and the server displays the final result.

178. Write a TCP client-server application where the client sends employee records and the server sorts them based on salary before returning them.

179. Write a TCP client-server application where the client sends ten integers and the server returns only the prime numbers.

180. Write a TCP client-server application to implement a simple online attendance system where multiple clients mark attendance and the server stores the records.

Topic 10 – Linux Memory Management

Dynamic Memory Allocation

- 181.** Write a program to dynamically allocate memory for an integer array using malloc() and calculate its sum.
- 182.** Write a program to allocate memory using calloc() for storing employee records and display the stored data.
- 183.** Write a program to resize an integer array using realloc() after inserting additional elements.
- 184.** Write a program to dynamically allocate memory for a matrix and perform matrix addition.
- 185.** Write a program to allocate memory for student records using structures and pointers.

Memory Mapping

- 186.** Write a program to copy the contents of one file to another using mmap() instead of read() and write().
- 187.** Write a program to map a text file into memory using mmap() and count the number of words.
- 188.** Write a program to map a file into memory and replace every occurrence of a specified word with another word.

Shared Mapping

- 189.** Write a program where a parent and child process communicate using memory mapped files (mmap()).
- 190.** Write a program where two unrelated processes share data through a memory mapped file.

Heap Management

- 191.** Write a program to demonstrate heap expansion using sbrk() by allocating memory dynamically.
- 192.** Write a program to display the program break before and after dynamic memory allocation using brk() or sbrk().

Virtual Memory

- 193.** Write a program to demonstrate the concept of Copy-on-Write (COW) after a fork() operation by modifying shared data in the child process.
- 194.** Write a program to observe virtual memory behavior by creating multiple large dynamic arrays.

Memory Operations

195. Write a program to compare the performance of malloc() and calloc() for allocating memory to a large integer array.
196. Write a program to dynamically allocate memory for storing employee records and sort them based on employee ID.
197. Write a program to demonstrate memory leakage by intentionally omitting free() and explain the consequences.
198. Write a program to dynamically allocate memory for two matrices and perform matrix multiplication.
199. Write a program to dynamically allocate memory for storing a paragraph and count the frequency of each character.
200. Write a program to simulate a simple memory allocation system that allocates and deallocates memory blocks dynamically using linked lists.

